ctrlX Drive Oscilloscope Service

How-To Guide and Use Cases

DriveOscopeApp

October 5, 2026

Contents

1 Overview **2**

2 Prerequisites **2**

3 Data Layer Interface **3**

4 Capture Lifecycle **3**

5 Data Formats **4**
- 5.1 ScopeConfig (create request) . . . 4
- 5.2 ScopeCapture (recorded trace) . . . 4

6 Use Cases **5**
- 6.1 UC1: Create a capture . . . 5
- 6.2 UC2: Wait until the scope is armed . . . 5
- 6.3 UC3: Record and read one trace . . . 6
- 6.4 UC4: Record another trace with the same settings . . . 6
- 6.5 UC5: Monitor a capture . . . 7
- 6.6 UC6: Delete a capture . . . 7
- 6.7 UC7: Recover from a failed capture . . . 7

7 Using the Command Line Client **7**

8 Result Code Reference **8**

1 Overview

The Drive Oscilloscope service runs on a ctrlX CORE as a snap. It records oscilloscope traces from a ctrlX DRIVE through the Easy Automation Library and exposes the whole workflow on the ctrlX Data Layer. A client never talks to the drive directly: it creates a *capture*, fires the trigger, and reads back the recorded samples, all through Data Layer nodes.

- One capture configures the oscilloscope of one drive.
- Each capture is identified by a server-generated `capture_id` (UUID).
- A capture can record many traces: trigger, read, re-arm, trigger again.
- A capture exists until the client deletes it. Captures do not survive a service restart.
- Only one capture per drive IP address can exist at a time.

The data exchanged is defined by two FlatBuffers schemas in `designDocs/interface/`, which are the source of truth for field names and types:

Schema	Root type	Used for
`scope_config.fbs`	`scope_config.ScopeConfig`	Body of the create request
`scope_capture.fbs`	`DriveCapture.ScopeCapture`	Recorded trace returned by the service

The service registers both as Data Layer types, so tools that understand Data Layer types (for example the ctrlX CORE Data Layer view) can show the capture as structured data:

Type address	Schema
`types/ctrlx-drive-oscope/scope-config`	`ScopeConfig`
`types/ctrlx-drive-oscope/scope-capture`	`ScopeCapture`

2 Prerequisites

- The service snap is installed and running on the ctrlX CORE. It needs no configuration: it always publishes under `ctrlx-drive-oscope/captures`.
- The drive is reachable from the ctrlX CORE over Ethernet at a fixed IPv4 address.
- The client has Data Layer access to the ctrlX CORE (an app running on the CORE, or a remote client with user credentials).
- For .NET clients: the `ctrlX-DataLayer` NuGet package from the ctrlX Automation SDK, and C# classes generated from the two schemas with `flatc` (see `src/DriveOscope/build-fbs.sh`).

Connecting from .NET. `new Remote().ToString()` picks the connection automatically: inside a snap on the ctrlX CORE it returns `ipc://`; anywhere else it builds a TCP connection string using the given IP address and credentials (factory defaults `192.168.1.1`, `boschrexroth`/`boschrexroth`, SSL port 443).

```
using var system = new DatalayerSystem();
system.Start(startBroker: false);   // a broker already runs on the ctrlX CORE

string remote = new Remote(ip: "192.168.1.1", user: "boschrexroth",
                           password: "boschrexroth", sslPort: 443).ToString();
using IClient client = system.Factory.CreateClient(remote);
if (!client.IsConnected) { /* handle: not connected */ }
```

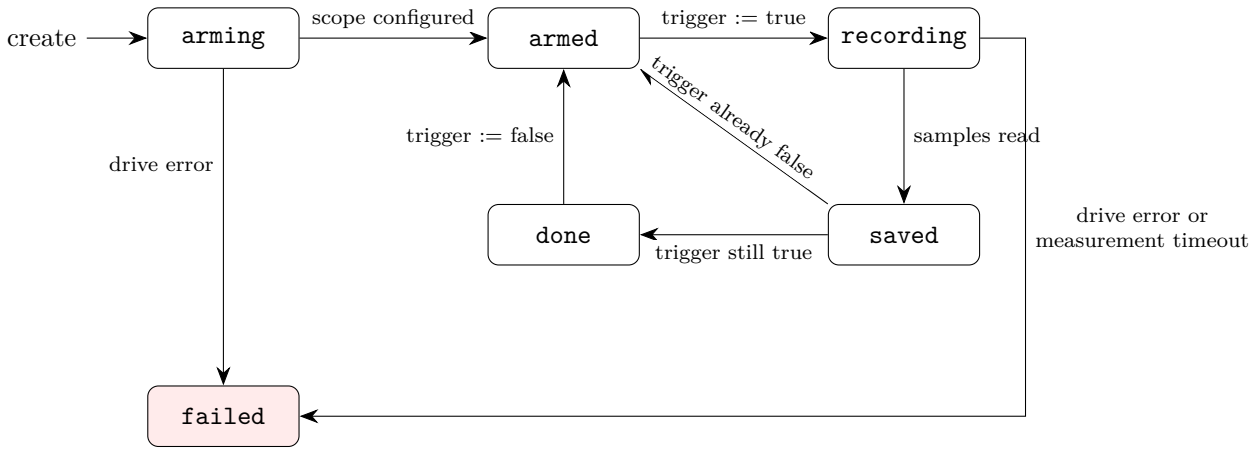
3 Data Layer Interface

All endpoints are below `ctrlx-drive-oscope/captures`. In the table, `{id}` is the `capture_id` returned when the capture was created.

Address (below `captures`)	Operation	Type	Meaning
`captures`	Create	`ScopeConfig`	Create a capture; reply is the new capture's address
`captures`	Browse	–	List of existing `capture_id`s
`{id}`	Remove	–	Delete the capture (fire and forget)
`{id}`	Browse	–	List of the leaf nodes below
`{id}/ready`	Read	bool	`true` while the scope is armed and waiting for the trigger
`{id}/trigger`	Read, Write	bool	Write `true` to fire, `false` to re-arm
`{id}/done`	Read	bool	`true` when the last trace has finished
`{id}/capture`	Read	`ScopeCapture`	The finished trace; fails until `done`
`{id}/state`	Read	string	Lifecycle state (Section 4)
`{id}/error`	Read	string	Failure reason; empty unless `state` is `failed`

Every request is answered immediately. The service never holds a Data Layer request open while it waits for the drive; long-running work happens in the capture's background worker, and the client polls `ready`, `done` and `state` to follow it.

4 Capture Lifecycle



A `DELETE` (Data Layer *remove*) is accepted in every state, including `failed`, and ends the capture.

State	What is happening	`ready`	`done`
`arming`	Connecting to the drive and writing the scope settings	false	false
`armed`	Scope running, waiting for the trigger	true	false
`recording`	Trigger fired; waiting for the scope memory to fill, then reading samples	false	false
`saved`	Samples read; result being stored (short, transient)	false	false
`done`	Trace available on `capture`	false	true
`failed`	Unrecoverable error or timeout; reason on `error`	false	false

Rules:

- `measurement_timeout_ms` is measured from the trigger. If the trace is not complete by then, the capture moves to `failed`.
- `failed` is final. Nothing is cleaned up automatically; the capture (and its drive reservation) stays until the client deletes it.

- After writing `trigger = false` in `done`, the service re-arms the scope. `ready` becomes `true` once the drive confirms; until then the state is still `done`.

5 Data Formats

5.1 ScopeConfig (create request)

Field	Type	Meaning and validation
`drive_ip_address`	string	Drive IPv4 address, four dotted octets.
`signals`	[string]	One SERCOS IDN per scope channel, form `S-x-xxxx` or `P-x-xxxx`. At least one.
`time_resolution_us`	uint32	Sample interval in µs (drive P-0-0031). Must be > 0.
`memory_size`	uint32	Number of samples per channel (drive P-0-0032). Must be > 0.
`pre_trigger_percent`	ubyte	Share of the memory recorded before the trigger, 0–100.
`measurement_timeout_ms`	uint32	Time allowed from trigger to finished trace. Must be > 0.

Example, shown as JSON (the format used by `DriveOscopeClient`; on the Data Layer the body is the `ScopeConfig` FlatBuffer):

```
{
  "drive_ip_address": "192.168.0.1",
  "signals": [ "S-0-0040", "S-0-0051" ],
  "time_resolution_us": 4000,
  "memory_size": 4096,
  "pre_trigger_percent": 20,
  "measurement_timeout_ms": 60000
}
```

5.2 ScopeCapture (recorded trace)

Field	Type	Meaning
`captured_at`	string	Time of the trigger, ISO 8601 with UTC offset.
`drive_ip_address`	string	Drive the trace came from.
`time_resolution_us`	uint32	Sample interval used.
`memory_size`	uint32	Memory size used.
`pre_trigger_percent`	ubyte	Pre-trigger share used.
`pre_trigger_samples`	uint32	Samples actually recorded before the trigger (see below).
`drive_p0032`	uint32	P-0-0032 read back from the drive after configuration.
`drive_p0033`	uint32	P-0-0033 read back from the drive after configuration: the number of samples recorded after the trigger.
`time_us`	[int32]	Time of each sample relative to the trigger; negative values are before the trigger.
`channels`	[Channel]	One entry per configured signal.
`Channel.index`	ubyte	Channel number, starting at 1, in the order of `signals`.
`Channel.signal`	string	SERCOS IDN recorded on this channel.
`Channel.values`	[float]	One value per entry in `time_us`. Up to `memory_size` values (see below).

Number of samples and the time axis. The drive returns only the samples it actually recorded. The part after the trigger is always complete (`drive_p0033` samples). The part before the trigger is only complete if the scope was armed long enough before the trigger, that is at least $(\text{memory_size} - \text{drive_p0033}) \cdot \text{time_resolution_us}$. A trigger fired sooner gives a shorter trace with less pre-trigger history. Therefore

$$\text{pre_trigger_samples} = N - \text{drive_p0033}$$

$$\text{time_us}[i] = (i - \text{pre_trigger_samples}) \cdot \text{time_resolution_us}, \qquad i = 0, \ldots, N-1$$

where N is the number of samples returned, so the sample at the trigger has time 0. Example with `memory_size` 4096, 20 % pre-trigger and 4 ms resolution (`drive_p0033` = 3276): triggering right after arming returned 3311 samples (35 before the trigger); triggering 12 s after arming returned all 4096 (820 before the trigger).

To get the full pre-trigger history, wait at least that long after `ready` becomes `true` before writing the trigger (UC3).

6 Use Cases

The code fragments use the .NET Data Layer client set up in the Prerequisites section. `captureAddress` is the value returned by the create request, for example:

`ctrlx-drive-oscope/captures/7e85ebde-8b45-4f59-afce-acbfb33a446d`

6.1 UC1: Create a capture

Goal: Configure the oscilloscope of one drive.

Precondition: No other capture exists for the same drive IP address.

Postcondition: A capture exists in state `arming`; its address is known.

Steps

1. Build a `ScopeConfig` FlatBuffer.
2. Send a Data Layer *create* on `ctrlx-drive-oscope/captures` with the FlatBuffer as argument.
3. On `DL_OK`, the reply value is the new capture's address (`ctrlx-drive-oscope/captures/{id}`). Keep it for all later requests.

```
var config = new ScopeConfigT {
    DriveIpAddress = "192.168.0.1",
    Signals = ["S-0-0040", "S-0-0051"],
    TimeResolutionUs = 4000, MemorySize = 4096,
    PreTriggerPercent = 20, MeasurementTimeoutMs = 60000,
};
var builder = new FlatBufferBuilder(Variant.DefaultFlatbuffersInitialSize);
builder.Finish(ScopeConfig.Pack(builder, config).Value);

var (result, location) = client.Create("ctrlx-drive-oscope/captures",
                                       new Variant(builder));
if (result.IsBad()) { /* DL_FAILED: invalid config or drive already in use */ }
string captureAddress = location.ToString();
```

Failure: `DL_FAILED` when the config breaks a validation rule, when the body is not a `ScopeConfig` FlatBuffer, or when another capture already uses the drive. The capture is not created. The service log states the exact reason.

Note: the create returns before the drive is contacted. A wrong but well-formed drive address is only detected afterwards; see UC2.

6.2 UC2: Wait until the scope is armed

Goal: Know when the trigger can be fired.

Precondition: UC1 succeeded.

Postcondition: `ready` is `true`, state `armed`.

Steps

1. Poll `{id}/ready` (for example every 200 ms).
2. On every poll where `ready` is `false`, also read `{id}/state`. If it is `failed`, read `{id}/error`, report it and go to UC6.
3. Use a client-side time limit (for example 30 s) for drives that never answer.

```
while (true) {
    var (r, ready) = client.Read($"{captureAddress}/ready");
    if (r.IsGood() && ready.ToBool()) break;
    var (s, state) = client.Read($"{captureAddress}/state");
    if (s.IsGood() && state.ToString() == "failed") { /* read error, delete */ }
    await Task.Delay(200);
}
```

6.3 UC3: Record and read one trace

Goal: Fire the trigger and get the recorded samples.

Precondition: State `armed` (UC2).

Postcondition: State `done`; the trace has been read.

Steps

1. For a full pre-trigger history, wait at least (memory_size − drive_p0033) · time_resolution_us after `ready` became `true` (Section 5.2).
2. Write `true` to `{id}/trigger`. The service raises the scope's manual trigger; `captured_at` is set to this moment.
3. Poll `{id}/done` until it is `true`, checking `state` for `failed` as in UC2. Allow at least `measurement_timeout_ms` plus a margin for reading the samples.
4. Read `{id}/capture` and decode it as a `ScopeCapture`. Reading is repeatable while the state is `done`.

```
client.Write($"{captureAddress}/trigger", new Variant(true));
// ... poll {id}/done until true ...
var (r, value) = client.Read($"{captureAddress}/capture");
ScopeCaptureT trace = ScopeCapture.GetRootAsScopeCapture(value.ToFlatbuffers()).UnPack
    ();
foreach (ChannelT channel in trace.Channels)
    Console.WriteLine($"{channel.Signal}: {channel.Values.Count} samples");
```

Failures:

- Writing the trigger returns `DL_FAILED` while the state is `arming` or `failed`: the scope settings are not on the drive.
- The drive reports a scope error, or the trace is not complete within `measurement_timeout_ms`: the state becomes `failed`.
- Reading `capture` before `done` returns `DL_FAILED`.

6.4 UC4: Record another trace with the same settings

Goal: Re-arm without creating a new capture.

Precondition: State `done` (UC3), trace read if needed.

Postcondition: State `armed`; continue with UC3.

Steps

1. Write `false` to `{id}/trigger`.
2. Wait for `ready` as in UC2. `done` becomes `false` when the scope is armed again.

3. Continue with UC3.

Important: re-arming discards the previous trace. Read `{id}/capture` before writing `false`.

Variant: if `false` is written while the trace is still recording, the service skips `done` and goes straight back to `armed` after the samples are read. The trace of that recording cannot be read.

6.5 UC5: Monitor a capture

Goal: Show the progress of a capture, for example in an HMI.

Steps

- Read `{id}/state` for the current lifecycle state.
- Read `{id}/trigger` for the last trigger value written.
- Browse `ctrlx-drive-oscope/captures` to list all existing captures.

6.6 UC6: Delete a capture

Goal: Stop the capture and free the drive.

Precondition: The capture exists (any state).

Postcondition: The capture's nodes are gone; the drive is released shortly after.

Steps

1. Send a Data Layer *remove* on the capture address.
2. `DL_OK` confirms the request. The capture disappears from the Data Layer immediately; further requests on its address return `DL_INVALID_ADDRESS`.

```
DLR_RESULT removed = client.Remove(captureAddress);
```

Behaviour: the delete is fire and forget. The service answers at once and the capture's worker stops and closes the drive connection in the background. Until that has finished (normally well under a second, at most about 8 s if the drive does not respond) the drive stays reserved, and a create for the same drive returns `DL_FAILED`. Retry the create after a short delay.

Always delete captures you no longer need, including failed ones. Nothing is cleaned up automatically, and a capture blocks its drive for as long as it exists.

6.7 UC7: Recover from a failed capture

Goal: Find out why a capture failed and try again.

Precondition: `{id}/state` reads `failed`.

Steps

1. Read `{id}/error` for the reason (for example a drive connection error, a parameter that could not be written, or a measurement timeout).
2. Delete the capture (UC6).
3. Fix the cause (drive address, signal IDNs, timeout) and create a new capture (UC1).

7 Using the Command Line Client

`src/DriveOscopeClient` runs UC1 to UC4 and UC6 from a PC. It reads a `ScopeConfig` as JSON (snake_case keys, as in the example above), creates the capture, waits until it is armed, fires the trigger when Enter is pressed, saves each trace as JSON, re-arms, and deletes the capture when it exits.

```
dotnet run --project src/DriveOscopeClient -- scopeConfig.json \
    --core-ip 192.168.1.1 --ssl-port 443 \
    --user boschrexroth --password boschrexroth --output output
```

All options are optional; the values shown are the defaults. Without a config path, `scopeConfig.json` next to the executable is used. Each trace is written to the output folder as `scope_yyyyMMdd_HHmmss.json`.

8 Result Code Reference

Request	Result	Cause
Create `captures`	`DL_OK`	Capture created; value is its address
	`DL_FAILED`	Invalid config, not a `ScopeConfig` FlatBuffer, or drive already in use
Create elsewhere	`DL_UNSUPPORTED`	Only `captures` accepts create
Read `{id}/...`	`DL_INVALID_ADDRESS`	Unknown or deleted `capture_id`, or unknown leaf
Read `{id}/capture`	`DL_FAILED`	State is not `done`
Write `{id}/trigger`	`DL_OK`	Accepted; value echoed back
	`DL_FAILED`	State is `arming` or `failed`
	`DL_INVALID_VALUE`	Value is not a bool (or `{"trigger": true}` JSON string)
Write elsewhere	`DL_UNSUPPORTED`	Only `trigger` is writable
Remove `{id}`	`DL_OK`	Delete accepted; cleanup runs in the background
	`DL_INVALID_ADDRESS`	Unknown or already deleted `capture_id`

Trigger value formats. The trigger accepts a Data Layer `bool`. For clients that can only send text, a string containing the JSON object `{ "trigger": true }` (or `false`) is accepted as well.